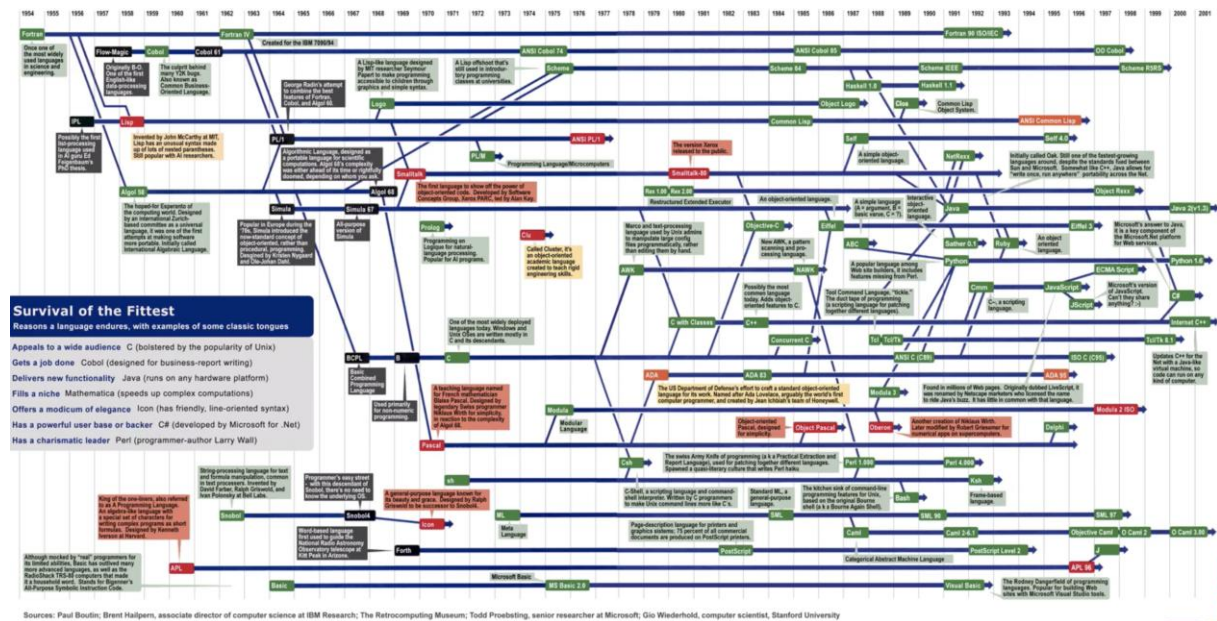


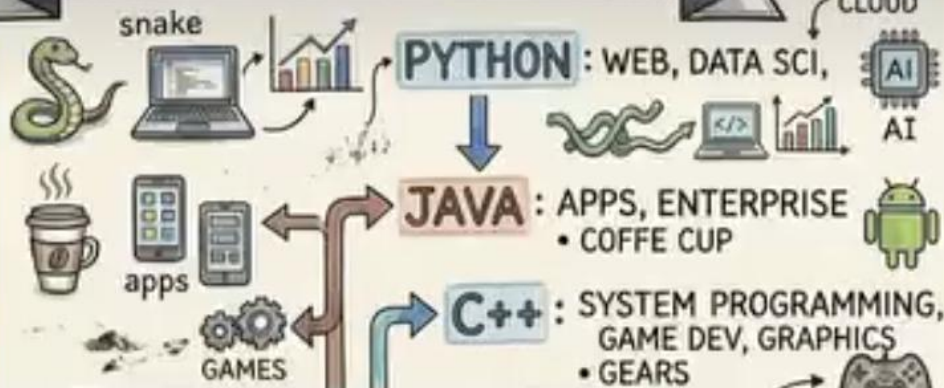
Motivation



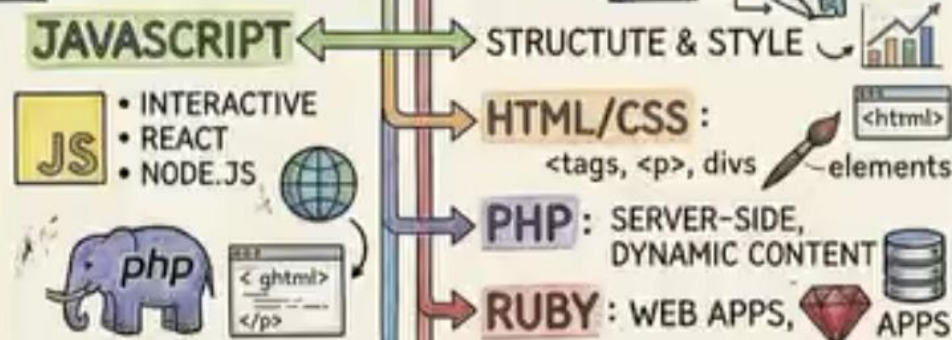
Source : <https://www.youtube.com/watch?v=PCGByjZpxIw>

TYPES OF CODING LANGUAGES & THEIR USES

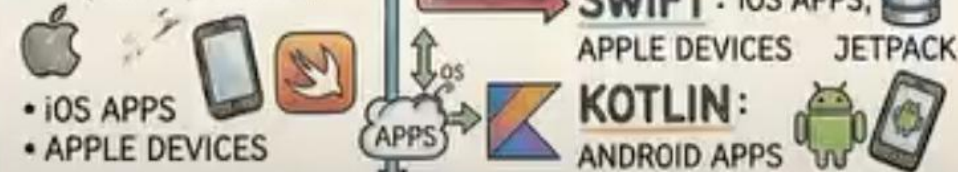
1. GENERAL PURPOSE & APPLICATIONS (PYTHON, JAVA, C++)



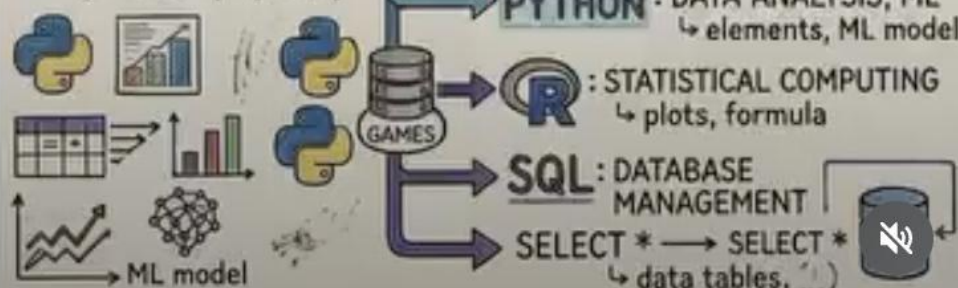
2. WEB DEVELOPMENT (JAVASCRIPT, HTML/CSS, PHP, RUBY)



3. MOBILE DEVELOPMENT (SWIFT, KOTLIN)



4. DATA SCIENCE & ANALYTICS (PYTHON, R, SQL)



Source : <https://www.instagram.com/popular/history-of-programming-languages-timeline-infographic/>

Le fichier pdf (généré par ChatGPT) est plus lisible.

Transparent 1 — Les paradigmes de programmation

Qu'est-ce qu'un paradigme de programmation ?

Un **paradigme de programmation** est une manière de concevoir et d'organiser un programme.

Il définit notamment :

- comment représenter les données ;
- comment exprimer les traitements ;
- comment organiser le programme ;
- comment raisonner sur son comportement.

Quatre paradigmes étudiés

Paradigme	Idée centrale
Structuré	Organiser le programme en instructions, conditions, boucles et fonctions
Orienté objet	Organiser le programme autour d'objets qui combinent données et comportements
Fonctionnel	Construire le programme à partir de fonctions et de transformations de données
Logique	Décrire des faits et des règles, puis laisser le système déduire les solutions

Transparent 2 — Pourquoi plusieurs paradigmes ?

Un même problème peut être exprimé de différentes manières

Problème :

Calculer la somme des nombres pairs d'une liste.

On peut :

- parcourir explicitement la liste → **programmation structurée** ;
- demander à un objet de réaliser le calcul → **programmation orientée objet** ;
- filtrer puis réduire la liste → **programmation fonctionnelle** ;
- décrire les propriétés permettant de déterminer le résultat → **programmation logique**.

👉 Le paradigme influence **la façon de penser le problème** avant même d'écrire le code.

1. Programmation structurée

Transparent 3 — Le paradigme structuré

Principe

Le programme est vu comme une **suite d'instructions** organisées en blocs.

Trois structures fondamentales :

Séquence

```
instruction 1
instruction 2
instruction 3
```

Sélection

```
SI condition
    ALORS ...
SINON
    ...
```

Répétition

```
TANT QUE condition
    ...
```

On utilise également des **fonctions/procédures** pour décomposer le programme.

Idée clé

Décomposer un problème complexe en sous-problèmes plus simples.

Transparent 4 — Exemple : programmation structurée

```
nombres = [4, 7, 2, 9, 6, 3]

somme = 0

for nombre in nombres:
    if nombre % 2 == 0:
        somme = somme + nombre

print(somme)
```

Résultat :

12

Caractéristiques

- instructions exécutées dans un ordre déterminé ;
- variables modifiées au cours du programme ;
- utilisation de conditions et de boucles ;
- fonctions pour structurer le code.

Langages

C, Pascal, Python, Ada, etc.

2. Programmation orientée objet

Transparent 5 — Le paradigme orienté objet

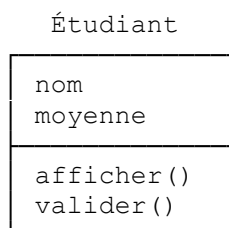
Principe

Le programme est organisé autour d'**objets**.

Un objet possède :

- un **état** → données/attributs ;
- un **comportement** → méthodes.

Exemple :



Concepts fondamentaux

- classe
- objet
- attribut
- méthode
- encapsulation
- héritage
- polymorphisme

Transparent 6 — Exemple en Python

```
class Etudiant:

    def __init__(self, nom, note):
        self.nom = nom
        self.note = note

    def est_admis(self):
        return self.note >= 10
```

```
etudiant = Etudiant("Alice", 15)
```

```
print(etudiant.nom)
print(etudiant.est_admis())
```

Ici :

Classe	→ Etudiant
Objet	→ etudiant
Attribut	→ nom, note
Méthode	→ est_admis()

Idée clé

Regrouper les données et les opérations qui les manipulent.

3. Programmation fonctionnelle

Transparent 7 — Le paradigme fonctionnel

Principe

Le programme est construit comme une **composition de fonctions**.

Une fonction :

entrée → fonction → sortie

On cherche autant que possible à utiliser des fonctions qui ne modifient pas leur environnement.

Concepts importants

- fonctions comme valeurs ;
- fonctions de haut niveau ;

- immutabilité ;
- composition de fonctions ;
- map, filter, reduce ;
- fonctions lambda.

Idée clé

Décrire ce que l'on veut calculer plutôt que détailler toutes les étapes de modification de l'état.

Transparent 8 — Exemple fonctionnel

Même problème :

Calculer la somme des nombres pairs.

```
nombre = [4, 7, 2, 9, 6, 3]

resultat = sum(
    filter(lambda x: x % 2 == 0, nombre)
)

print(resultat)
```

On peut décomposer le traitement :

```
liste
  ↓
filter
  ↓
nombre pairs
  ↓
sum
  ↓
résultat
```

Ou, en Python, utiliser une compréhension :

```
resultat = sum(x for x in nombre if x % 2 == 0)
```

4. Programmation logique

Transparent 9 — Le paradigme logique

Principe

On décrit :

1. des **faits** ;
2. des **règles** ;
3. des **requêtes**.

Le programme cherche ensuite une solution par **déduction logique**.

Exemple

```
parent(alice, bob).
parent(bob, charlie).

grand_parent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
```

Question :

```
?- grand_parent(alice, charlie).
```

Réponse :

```
true
```

Idée clé

Décrire les connaissances et les relations plutôt que les étapes de calcul.

Langage emblématique

Prolog

5. Comparaison

Transparent 10 — Les quatre paradigmes

	Structuré	Orienté objet	Fonctionnel	Logique
Unité principale	Instruction / fonction	Objet	Fonction	Fait / règle
État	Variable modifiable	Encapsulé dans les objets	Limité / évité	Implicite
Contrôle	Séquence, condition, boucle	Messages / méthodes	Composition	Inférence
Question centrale	Comment ?	Qui fait quoi ?	Quelle transformation ?	Quelles règles ?
Exemple	C, Python	Java, C++, Python	Haskell, Lisp, Python	Prolog

Transparent 11 — Un même problème, quatre visions

Exemple : gérer les étudiants d'une université

Structuré

```
lire les étudiants
→ parcourir la liste
→ calculer les moyennes
→ afficher les résultats
```

Orienté objet

```
Université
  ↓
Étudiants
  ↓
Etudiant
  ↓
calculer_moyenne()
```

Fonctionnel

```
étudiants
  ↓
map(calculer_moyenne)
  ↓
filter(admis)
  ↓
résultat
```

Logique

```
étudiant(alice).
note(alice, 15).

admis(X) :- note(X, N), N >= 10.
```

Transparent 12 — Les paradigmes ne sont pas exclusifs

Dans la pratique, les langages modernes combinent plusieurs paradigmes.

Python, par exemple, permet :

- programmation structurée ;
- programmation procédurale ;
- programmation orientée objet ;
- programmation fonctionnelle.

On peut donc écrire dans un même programme :

```
class Etudiant:  
    ...
```

et utiliser :

```
filter(...)  
map(...)  
lambda
```

tout en ayant :

```
for ...  
if ...
```

Message important

Un paradigme est une manière de penser et de structurer un programme, pas simplement une caractéristique d'un langage.

Transparent 13 — À retenir

Programmation structurée

Décomposer le programme en instructions, structures de contrôle et fonctions.

Programmation orientée objet

Modéliser le problème avec des objets qui possèdent un état et des comportements.

Programmation fonctionnelle

Construire le programme par composition et transformation de données avec des fonctions.

Programmation logique

Décrire des faits et des règles pour permettre au système de déduire les solutions.

Question finale pour les étudiants

Comment exprimeriez-vous le même problème avec chacun de ces quatre paradigmes ?

Cela peut constituer une bonne transition vers un **TP où les étudiants résolvent un même problème en programmation structurée, orientée objet et fonctionnelle**, puis découvrent un petit exemple en Prolog.

Pour aller plus loin :

Histoire et taxonomie des langages de programmation :

- <https://www.fun-mooc.fr/fr/cours/numerique-et-sciences-informatiques-les-fondamentaux/>
- <https://www.youtube.com/watch?v=PCGByjZpxIw>
- https://www.instagram.com/reel/DWDhbWbjmEB/?utm_source=popular_topic_grid

The [International Obfuscated C Code Contest](#) (IOCCC) is the famous annual programming competition where coders write the most unreadable, bizarre, and confusing yet working C code possible.